

Combining Q-Learning and Search with Amortized Value Estimates

-- ICLR2020

Hongming Zhang
2022.01.26

Introduction



- Search with Amortized Value Estimates (SAVE) combines model-free **Q-learning** with model-based **Monte-Carlo Tree Search** (MCTS).
- Use **perfect model** of the environment.
- Amortized Value Estimates: amortize the value computation performed by MCTS.

Motivation



Pros and cons:

- Model-based methods hold the promise of allowing agents to more flexibly **adapt to new situations** and efficiently **reason**.
- Planning **avoids suboptimal actions**, but information about suboptimal actions therefore does **not get propagated** back to the Q-function.

Goal

- Combine planning with model-free methods, and also propagate information about suboptimal actions.

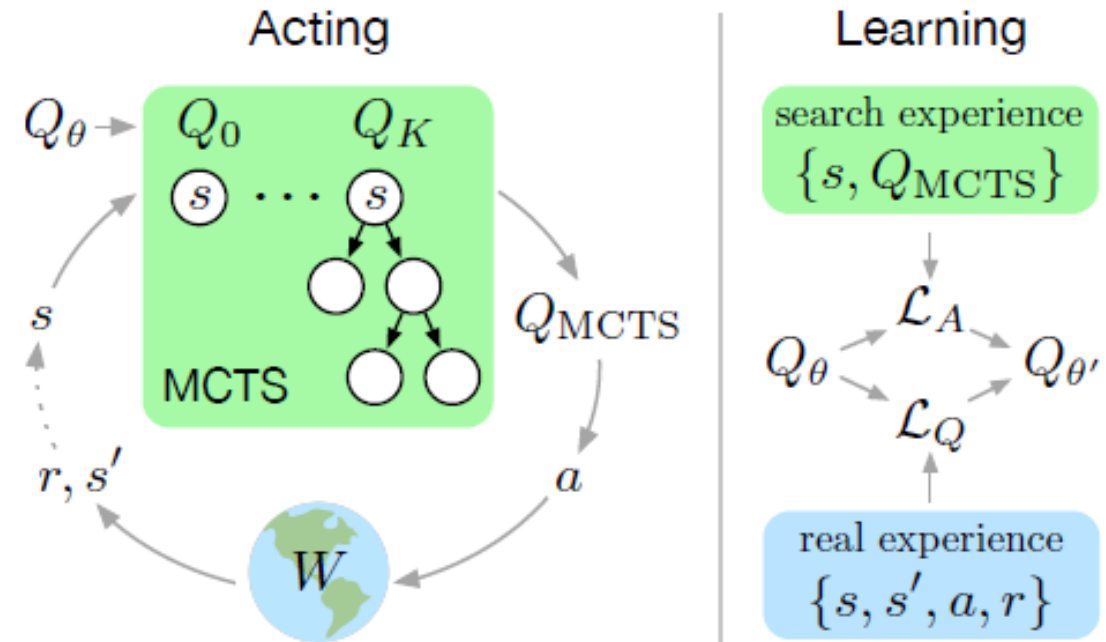
Algorithm details

Search/Planning (Acting)

- Search policy that incorporates the Q-function as a **prior over Q-values**. After search, select an action via epsilon-greedy over $Q_{MCTS}(s_0, a)$.

Learning

- An objective function that combines both the **TD-error** from Q-learning with an **amortization loss** that amortizes the value computation performed by the search



Algorithm details

- Standard MCTS

$$\pi_k(s) = \operatorname{argmax}_a (Q_k(s, a) + U_k(s, a))$$

$$Q_k(s, a) = \sum_{i=1}^{N_k(s, a)} R_i(s, a) / N_k(s, a)$$

$$R_i(s_t, a_t) = \sum_{j=t}^{T-1} \gamma^{j-t} r_j + \gamma^{T-t} V(s_T)$$

Value $V(s_T)$ is estimated either via a state-value function or Monte-Carlo rollout

$$U_k(s, a) = c_{UCT} \sqrt{\frac{\log(\sum_a N_k(s, a))}{N_k(s, a)}}$$

No prior probability in $U_k(s, a)$

Algorithm details



- Search (Acting)

Search policy that incorporates the Q-function as a **prior** over Q-values.

1) Assumes it has visited every state and action pair once by initializing $N(s, a) = 1$ for all states and action.

2) Uses $Q_\theta(s, a)$ as an initial estimate for Q_k .

$$Q_0(s, a) := Q_\theta(s, a) \text{ and } Q_k(s, a) = \frac{Q_\theta(s, a) + \sum_{i=1}^{N_k(s, a)-1} R_i(s, a)}{N_k(s, a)}.$$

3) $V(s) := \max_a Q_\theta(s, a)$

Algorithm details



- Learning

An objective function that combines both the **TD-error** from Q-learning with an **amortization loss** that amortizes the value computation performed by the search

$$\mathcal{L}(\theta, \mathcal{D}) = \beta_Q \mathcal{L}_Q(\theta, \mathcal{D}) + \beta_A \mathcal{L}_A(\theta, \mathcal{D})$$

TD updates

1-step TD targets,
n-step TD targets,
 λ -returns

Cross-entropy

$$\mathcal{L}_A(\theta, \mathcal{D}) = -\frac{1}{N} \sum_{\mathcal{D}} (P_{MCTS})^T \log P_{\theta}$$

$$P_{MCTS} = \text{softmax}_{\tau}(Q_{MCTS}(s, \cdot))$$

$$P_{\theta} = \text{softmax}_{\tau}(Q_{\theta}(s, \cdot))$$

Algorithm details

Algorithm A.1 Pseudocode for the SAVE algorithm.

```
1: procedure SAVE( $\theta$ )
2:   while true do
3:     Begin episode at  $s$ 
4:     while acting do
5:       Estimate  $Q_{\text{MCTS}}(s, \cdot) \leftarrow \text{MCTS}(s, Q_\theta)$ 
6:       Select  $a$  using epsilon-greedy from  $Q_{\text{MCTS}}(s, \cdot)$ 
7:       Execute  $a$  in environment and receive  $s', r$ 
8:       Add  $(s, a, r, s', Q_{\text{MCTS}}(s, \cdot))$  to replay buffer
9:        $s \leftarrow s'$ 
10:    while learning do
11:      Sample minibatch of experience from the replay buffer
12:      Update  $\theta$  to minimize Equation 6
13:
14: procedure MCTS( $s_0, Q_\theta$ )
15:    $Q_0(s, a) \leftarrow Q_\theta(s, a)$  for all  $s, a$ 
16:    $N_0(s, a) \leftarrow 1$  for all  $s, a$ 
17:    $k \leftarrow 0$ 
18:   while search budget remains ( $k < K$ ) do
19:     Traverse the search tree with  $\pi_k$  (Equation 1)
20:     Expand new state  $s_T$  and add it to the search tree
21:     Evaluate  $\max_a Q_\theta(s_T, a)$  and backup returns (Equation 3)
22:     Set  $N_{k+1}(s, a) \leftarrow N_k(s, a)$  and then increment counts of visited states and actions
23:     Compute estimates for  $Q_{k+1}(s, a)$  (Equation 4)
24:      $k \leftarrow k + 1$ 
25:   Return  $\{Q_K(s_0, a_i)\}_i$ 
```

Experiment and analysis

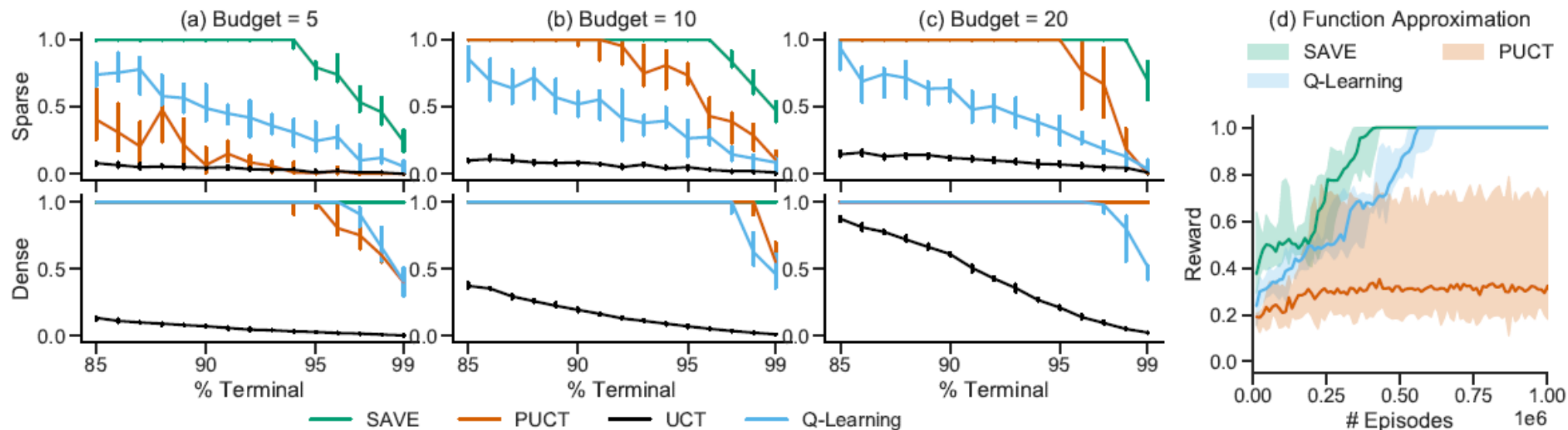


Figure 2: Results on Tightrope. (a-c) Tabular results comparing SAVE, PUCT, UCT, and Q-learning (with MCTS at test time) for varying percentages of terminal actions on the x -axes and for different search budgets. The y -axes show reward for either the sparse or dense reward setting of Tightrope. Lines show medians across 20 seeds, with error bars showing 95% confidence intervals. (d) Results on Tightrope when using function approximation, comparing SAVE with PUCT and Q-learning. Lines show medians across 10 seeds, with shaded regions indicating min and max seeds.

Experiment and analysis



- SAVE: act by MCTS with $Q_{MCTS}(s_0, a)$ during testing

- PUCT: AlphaZero style $\pi_k(s) = \operatorname{argmax}_a (Q_k(s, a) + U_k(s, a))$

$$U_k(s, a) = c_{UCT} \cdot \pi(s, a) \sqrt{\frac{\log(\sum_a N_k(s, a))}{N_k(s, a)}}$$

$$\pi(s, a) = (1 - \epsilon) \cdot \pi_\theta(s, a) + \epsilon \eta, \text{ Dirichlet noise } \eta \sim \text{Dir}(1/n_{\text{actions}})$$

$$\mathcal{L}_Q = \frac{1}{N} \sum_{\mathcal{D}} \|V_\theta(s) - R\|_2, \text{ Monte-Carlo return}$$

$$\mathcal{L}_A = -\frac{1}{N} \sum_{\mathcal{D}} (\pi_{MCTS}(s, \cdot))^T \log \pi_\theta(s, \cdot), \text{ visit counts}$$

- UCT: pure UCT without network $\pi_k(s) = \operatorname{argmax}_a (Q_k(s, a) + U_k(s, a))$

$$U_k(s, a) = c_{UCT} \sqrt{\frac{\log(\sum_a N_k(s, a))}{N_k(s, a)}}$$

- Q-Learning: without search during training and testing

Experiment and analysis

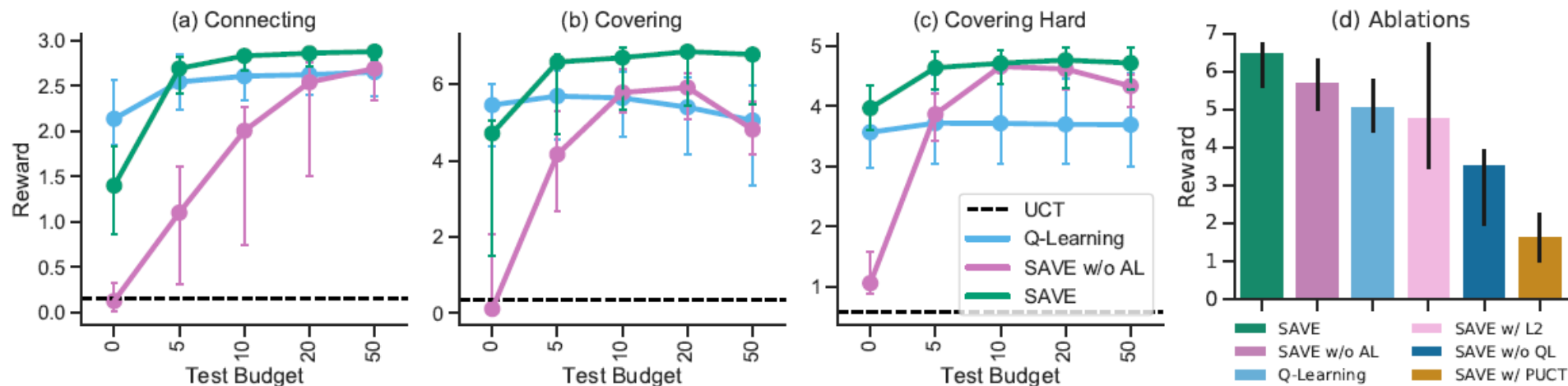


Figure 3: Results on Construction. (a-c) Each subplot shows results for SAVE, SAVE without amortization loss, Q-learning with MCTS at test time, and pure search (UCT). The x -axis shows the effect of increasing the number of MCTS simulations at test time. During training, SAVE with and without amortization loss used a search budget of 10 simulations. UCT used a search budget of 1000 simulations. Points show medians across 10 seeds, and error bars indicate min and max seeds. (d) Ablation experiments on the Covering task. We compare SAVE to variants that do not have an amortization loss, which use an L2 amortization loss, which do not use the Q-Learning loss, and which use PUCT rather than UCT. Results are shown at the hardest level of difficulty for the Covering task with a test budget of 10. The colored bars show median reward across 10 seeds, and error bars show min and max seed.

Conclusion



- SAVE reaches **higher performance** than model-free approaches while using **less computation** than is required by other model-based methods.
- SAVEing the experience generated by search in an explicit Q-function, and initializing future searches with that information, offers important advantages for model-based RL.

Thanks!

	Training	test	
MCTS+DQN (MCTS)	True	True	> DQN (argmax Q)
	True	False	< DQN
DQN	False	True	> MCTS+DQN